

Generic Asynchronous FIFO

Design and Verification Report

Emir Uruç

September 2026

Contents

1. Overview	2
2. Motivation and Background	3
3. Architecture	4
3.1 Pointers and Memory	4
3.2 Gray-Code Conversion and Cross-Domain Synchronization	4
3.3 Full and Empty Detection	4
4. Verification	5
4.1 Testbench 1 — Basic Functional Test	5
4.2 Testbench 2 — Full-Flag Overwrite Protection	5
4.3 Testbench 3 — Simultaneous Write + Read	7
4.4 Verification Method	7
5. Implementation Constraints and Timing Closure	8
5.1 Constraints File (XDC)	8
5.2 Timing Failure: Root Cause	9
5.3 Solution and Result	9
6. Design Process Notes	10
Architectural References	10

1. Overview

This project implements a fully parametrizable **asynchronous FIFO** in VHDL, designed to safely transfer data across two independent, unrelated clock domains (`clk_wr` and `clk_rd`). The design is built around the industry-standard Gray-code pointer synchronization technique, is accompanied by three self-written testbenches that verify its core correctness properties, and was carried through synthesis, implementation, and timing closure on real device constraints.

The entity is generic over three parameters:

- `data_width` — width of a single FIFO word, in bits
- `addr_width` — address width; FIFO depth is 2^{**}addr_width
- `N` — number of synchronizer flip-flop stages used when crossing clock domains

Repository contents:

File	Purpose
<code>async_FIFO_generic.vhd</code>	The FIFO itself
<code>tb_async_FIFO_generic.vhd</code>	Testbench 1 — basic write/read functional test
<code>tb2_async_FIFO_generic.vhd</code>	Testbench 2 — full-flag overwrite protection test
<code>tb3_async_FIFO_generic.vhd</code>	Testbench 3 — simultaneous write+read stress test
<code>async_FIFO_generic.xdc</code>	Pin assignment and timing constraints for implementation

2. Motivation and Background

The design builds directly on clock-domain-crossing (CDC) fundamentals: a single-bit “flag” synchronizer (toggle flip-flop + multi-stage synchronizer + edge detection) that safely transfers a one-cycle pulse between two asynchronous clocks. An asynchronous FIFO extends this same idea to a **multi-bit, continuously changing value** — the read and write pointers — rather than a single event pulse.

The central problem this design solves: if a binary pointer’s bits were synchronized independently, a multi-bit transition (e.g. 0111 -> 1000) could be sampled mid-transition and produce a corrupted intermediate value on the receiving side, since several bits change at once. Gray-code encoding guarantees that only **one bit** changes between any two consecutive pointer values, which eliminates this class of error and reduces the problem back to the single-bit case already solved by the flag synchronizer.

3. Architecture

3.1 Pointers and Memory

Two independent pointers (`wr_ptr_bin`, `rd_ptr_bin`) track the next write/read location. Each pointer is `addr_width + 1` bits wide — one bit wider than strictly needed to address the RAM — so that full and empty conditions can be distinguished even when both pointers wrap around to the same RAM address. Only the lower `addr_width` bits of each pointer are used to index the memory array; the extra bit is used exclusively in the full/empty comparison logic.

The FIFO storage itself is a simple dual-access array (`ram_type`), written on `clk_wr` and read on `clk_rd`.

3.2 Gray-Code Conversion and Cross-Domain Synchronization

Each pointer is converted to Gray code (`wr_ptr_gray`, `rd_ptr_gray`) before being passed to the opposite clock domain. The crossing itself is implemented as an **N-stage synchronizer**, modeled as an array of registers (type `reg` is array (0 to N-1) of `unsigned(...)`) rather than a single concatenated shift-register vector. Each stage holds one complete Gray-coded pointer value; a for loop shifts the value through the stages on every clock edge. Stage N-1 is the input stage (fed directly from the source domain), stage 0 is the settled, safe-to-use output. All synchronizer registers carry the `ASYNC_REG` attribute, instructing the synthesis and placement tools to keep the stages physically close together and treat the path as a CDC path in timing analysis.

This array-based approach is a deliberate implementation choice, distinct from the more commonly seen single-long-vector shift-register style; both are functionally equivalent, but the array form keeps each pipeline stage individually addressable and, in the author's view, easier to reason about and debug.

3.3 Full and Empty Detection

- **Empty** (evaluated in the `clk_rd` domain): the local read pointer (Gray) equals the synchronized write pointer (Gray) — an exact match.
- **Full** (evaluated in the `clk_wr` domain): the local write pointer (Gray) equals the synchronized read pointer (Gray) with its **top two bits inverted** — the Gray-code equivalent of “the write pointer is exactly one lap ahead of the read pointer.”

Both flags are computed from the **registered** Gray pointers (not the combinational “next” values), which is a simpler variant of the classic look-ahead formulation; it was a deliberate simplification made and verified during this project rather than a direct copy of the reference architecture's timing.

4. Verification

Three testbenches were written to exercise the design from different angles. All use two independent, non-integer-related clock periods ($\text{clk_wr} = 100 \text{ MHz} / 10 \text{ ns}$, $\text{clk_rd} = 133.3 \text{ MHz} / 7.5 \text{ ns}$) to genuinely stress the CDC paths rather than relying on a convenient clock ratio.

4.1 Testbench 1 — Basic Functional Test

Writes a full FIFO (16 words, an incrementing $0x00..0x0F$ pattern) on clk_wr , then reads all 16 words back on clk_rd . This confirms:

- Data integrity — values are read back in the exact order they were written
- full asserts at precisely the correct write count, not early or late, and stays high until reads begin
- empty is high before the first write and asserts again at precisely the correct read count

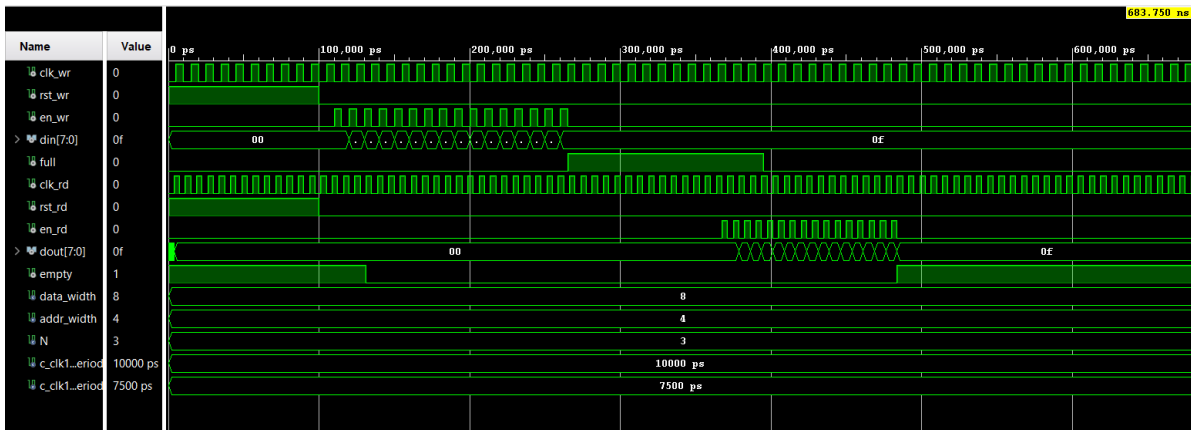


Figure 1: Testbench 1 - full write phase (16 words), full asserting and holding, followed by the read phase and empty asserting once all data has been consumed.

4.2 Testbench 2 — Full-Flag Overwrite Protection

Fills the FIFO to capacity (16 writes, $0x00..0x0F$), then attempts one additional write ($0xAA$) while $\text{full} = '1'$. The waveform shows full asserting immediately after the 16th write and the subsequent $0xAA$ write attempt failing to change dout or propagate into the FIFO:

As a second, lower-level check, the internal ram array and the write pointer were inspected directly in the waveform viewer after the rejected write: the pointer (wr_ptr_bin_next , frozen at 10) does not advance, and all sixteen RAM locations still hold their original $00..0F$ values — confirming the overwrite was blocked at the memory level, not just at the output port:

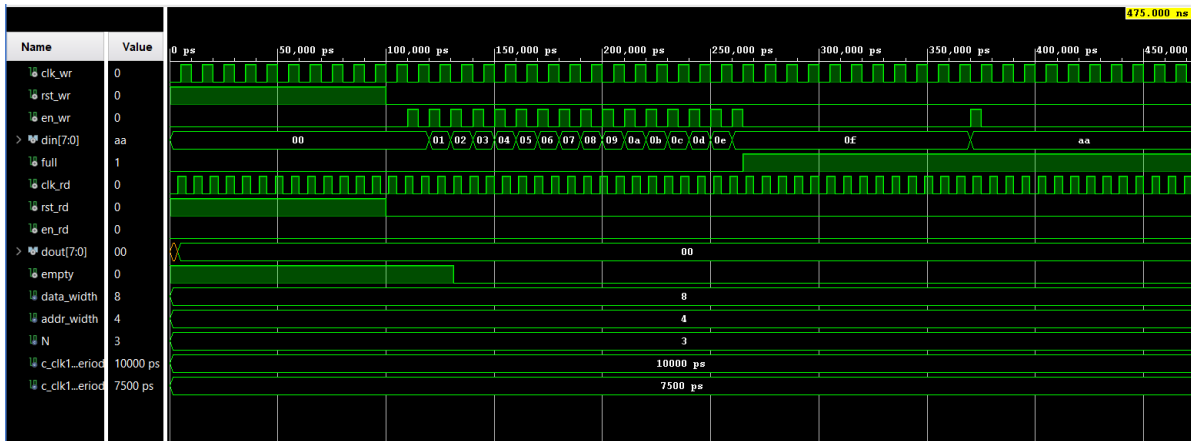


Figure 2: Testbench 2 - the FIFO fills to 0x0F, full asserts, and the later attempt to write 0xAA while full is rejected at the port level.

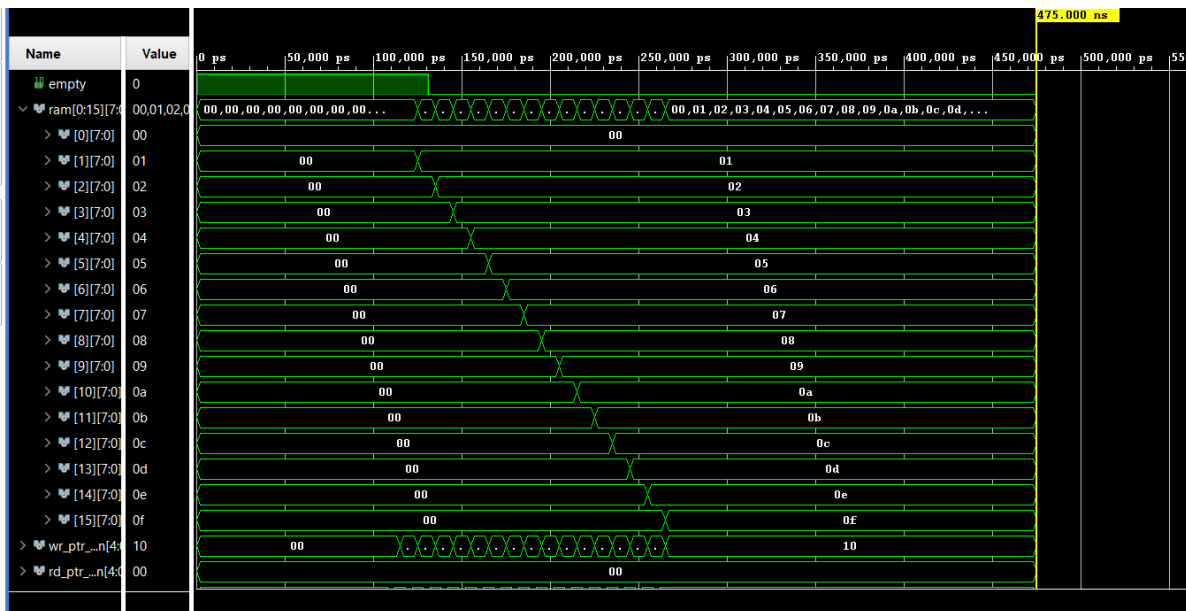


Figure 3: Testbench 2 (internal view) - after the rejected 0xAA write attempt, all 16 RAM locations still hold their original values and the write pointer is frozen.

4.3 Testbench 3 — Simultaneous Write + Read

Pre-loads the FIFO to a mid-depth state, then holds both en_wr and en_rd high concurrently for an extended window, allowing the write and read processes to run independently and repeatedly on their own clocks at the same time. This checks that:

- full and empty are never asserted simultaneously (a basic logical invariant of the design)
- Data already in the FIFO is read back correctly and in order while new data is being written
- The simulation completes cleanly with no unknown (X/U) values propagating through the design

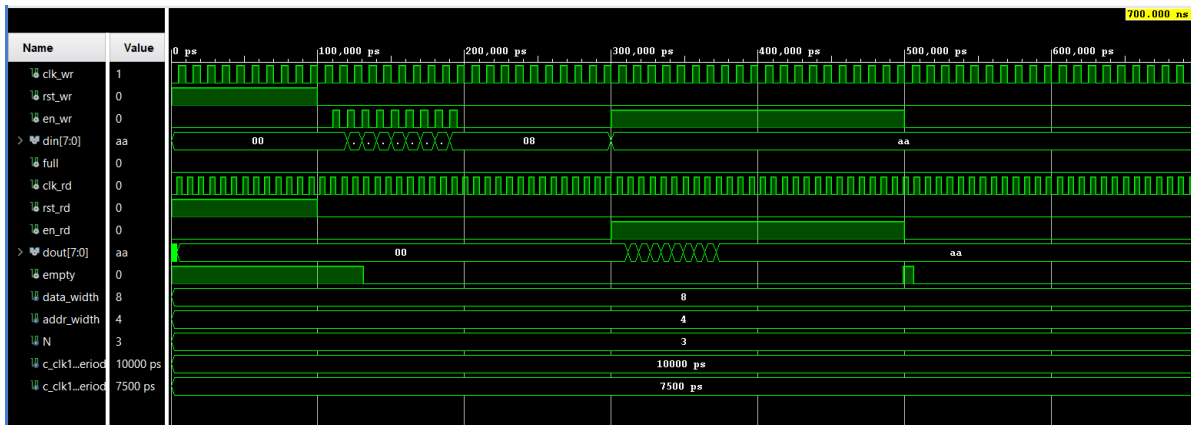


Figure 4: Testbench 3 - concurrent write and read activity at mid-depth; full and empty remain mutually exclusive throughout, and previously written data is read back correctly while new 0xAA data is written.

4.4 Verification Method

Verification relied on direct waveform inspection — including opening internal (non-port) signals such as the RAM array and pointer values inside the simulator hierarchy — rather than solely on port-level signals or scripted assertions. This approach was chosen deliberately as a learning exercise and proved effective at catching at least one real bug (an internal/external signal naming mismatch on the empty output) that was not apparent from reading the source code alone.

5. Implementation Constraints and Timing Closure

Beyond RTL simulation, the design was carried through synthesis and implementation on a real Xilinx target (pins assigned via Vivado's I/O Planning view, auto-placed), which surfaced a timing-closure issue specific to CDC designs and is documented here as part of the verification story.

5.1 Constraints File (XDC)

Pin locations were assigned using Vivado's I/O Planning / auto-place I/O flow rather than hand-picked; clock constraints were added manually to match the two independent clock periods used throughout simulation.

```
## Pin assignment (I/O Planning, auto-placed)
set_property PACKAGE_PIN V10 [get_ports en_rd]
set_property PACKAGE_PIN U11 [get_ports rst_wr]
set_property PACKAGE_PIN V12 [get_ports en_wr]
set_property PACKAGE_PIN U12 [get_ports rst_rd]
set_property PACKAGE_PIN T15 [get_ports full]
set_property PACKAGE_PIN U13 [get_ports {din[3]}]
set_property PACKAGE_PIN V11 [get_ports {din[7]}]
set_property PACKAGE_PIN U14 [get_ports {din[6]}]
set_property PACKAGE_PIN V14 [get_ports {din[5]}]
set_property PACKAGE_PIN V17 [get_ports {dout[1]}]
set_property PACKAGE_PIN T13 [get_ports {din[4]}]
set_property PACKAGE_PIN U17 [get_ports {dout[4]}]
set_property PACKAGE_PIN T9 [get_ports {din[2]}]
set_property PACKAGE_PIN T10 [get_ports {din[1]}]
set_property PACKAGE_PIN U16 [get_ports {dout[2]}]
set_property PACKAGE_PIN T11 [get_ports {dout[0]}]
set_property PACKAGE_PIN V16 [get_ports {dout[5]}]
set_property PACKAGE_PIN U18 [get_ports {dout[3]}]
set_property PACKAGE_PIN R10 [get_ports {din[0]}]
set_property PACKAGE_PIN T16 [get_ports {dout[7]}]
set_property PACKAGE_PIN N15 [get_ports clk_rd]
set_property PACKAGE_PIN R16 [get_ports empty]
set_property PACKAGE_PIN T14 [get_ports clk_wr]
set_property PACKAGE_PIN V15 [get_ports {dout[6]}]

## Clock definitions
create_clock -period 10.000 -waveform {0.000 5.000} [get_ports clk_wr]
create_clock -period 7.500 -waveform {0.000 3.750} [get_ports clk_rd]

## Declare the two clocks as unrelated (added to resolve the timing
## failure described in Section 5.2)
set_clock_groups -asynchronous \
    -group [get_clocks clk_wr] \
    -group [get_clocks clk_rd]
```

5.2 Timing Failure: Root Cause

The first implementation run, without the `set_clock_groups` constraint, failed timing with 6 failing endpoints and a worst negative slack (WNS) around **-0.14 to -0.20 ns**. The cause was not a functional CDC bug, but a static timing analysis (STA) artifact:

- The write pointer's RAM word (implemented as distributed RAM, RAMD32) is written on `clk_wr` and its output is registered on `clk_rd` on the read side.
- With no statement in the XDC file declaring `clk_wr` and `clk_rd` as unrelated, Vivado's STA engine defaulted to treating this RAM-bridged path as an ordinary **synchronous** path, and computed a setup requirement based on the closest coincident edges of the two clocks.
- Because the clock periods (10 ns and 7.5 ns) are not integer multiples of one another, the closest coincident-edge window Vivado found was only about **2.5 ns** — far shorter than the RAM's actual internal delay, so the path could never meet timing as analyzed.

Critically, this was **not** a sign of a broken design: the RTL-level CDC protection (Gray-code pointers passed through the N-stage, `ASYNC_REG`-tagged synchronizer) was already correct and verified in simulation. The failure was purely a case of the timing tool analyzing an asynchronous path as if it were synchronous, because it had not been told otherwise.

5.3 Solution and Result

Adding the following constraint tells Vivado explicitly that `clk_wr` and `clk_rd` are unrelated clock domains, which causes the tool to drop the (impossible) direct timing relationship between them and treat crossings between the two groups as false paths for STA purposes:

```
set_clock_groups -asynchronous \  
  -group [get_clocks clk_wr] \  
  -group [get_clocks clk_rd]
```

After re-running implementation with this constraint in place:

Metric	Before	After
WNS (Worst Negative Slack)	-0.203 ns	+4.240 ns
Failing endpoints	6	0

This resolved timing closure without any change to the RTL — confirming that the earlier failure was a constraints/STA issue rather than a design defect, and illustrating a point relevant to any CDC design: RTL-level correctness (Gray coding, synchronizer staging) and implementation-level timing closure (declaring clock relationships in the XDC) are two separate concerns that must both be addressed.

6. Design Process Notes

The design was developed iteratively, starting from an already-understood single-bit CDC flag synchronizer and generalizing it step by step to a full multi-bit, memory-backed FIFO:

1. Entity/port/generic definition
2. RAM declaration and addressing
3. Write-pointer arithmetic and Gray-code conversion
4. Full detection
5. Read-pointer arithmetic, Gray-code conversion, and empty detection
6. N-stage synchronizer design (array type + for loop), applied in both directions
7. Library/type consistency pass (standardizing on NUMERIC_STD / unsigned throughout)
8. Synthesis debugging and cleanup
9. Testbench development (three independent scenarios, described in Section 4)
10. Implementation, pin/clock constraints, and timing closure (Section 5)

Several design decisions were made independently during this process and are worth noting explicitly, since they diverge from the most commonly published reference implementation:

- The N-stage synchronizer is implemented as an array of registers with a for-loop shift, rather than a single concatenated bit-vector.
- full/empty are computed from registered Gray pointers rather than their combinatorial “next” values.
- Internal signals `full_o/empty_i` were introduced to work around VHDL’s restriction on reading back from out ports.

Architectural References

The overall architecture — Gray-code pointer synchronization, multi-stage synchronizers, and MSB-inversion-based full detection — follows the standard technique described in:

- C. E. Cummings, *“Simulation and Synthesis Techniques for Asynchronous FIFO Design,”* SNUG 2002.
- P. P. Chu, *RTL Hardware Design Using VHDL: Coding for Efficiency, Portability, and Scalability*, Wiley, 2006 (Chapter 16, FIFO design), consulted for general port-structuring ideas.
- P. P. Chu, *FPGA Prototyping by VHDL Examples*, Wiley, 2008, consulted as a general VHDL design and I/O-constraints reference.

The VHDL implementation itself — signal naming, process structure, the array-based synchronizer, and all design decisions listed above — is original work, written and debugged independently.